

EE3033 System Integration and Design Lab

Final Project (Group 10)

Name	Matriculation Number
Ewe Zi Heng	A0303237N
Jiang Yi'An	A0301377J
Ye Yucheng	A0302670N

1. Project objective

The objective is to equip a self-balancing robot with a ROS 2 software system that can map an unknown environment, autonomously search for an object placed after mapping, estimate the object's position in the map coordinate frame, and return to its starting area. The environment and target position are not pre-programmed. The design therefore requires a reusable mapping stage followed by a navigation and perception stage operating on the saved map.

1.1 System design

The implementation divides the task into sensor acquisition, simultaneous localization and mapping (SLAM), saved-map localization, route planning, visual detection, target-position estimation and mission coordination. These functions exchange data through ROS 2 topics and TF transformations. The mapping and autonomous stages are kept separate to match the operating procedure: the robot first constructs the map with permitted teleoperation, then uses that map for a self-directed search and return mission.

2. Overall system architecture

2.1 Hardware and software arrangement

The mobile platform comprises the balancing-car base controller, an N10P two-dimensional LiDAR and a USB camera. The robot-side ROS 2 Humble system runs inside the ros2_humble Docker container on the Raspberry Pi. An Ubuntu virtual machine provides RViz2 visualization, mission coordination and camera inference. Both machines use ROS 2 distributed communication on the same network and ROS_DOMAIN_ID 10 in the recorded configuration.

The base-control node receives geometry_msgs/Twist commands from /cmd_vel and publishes robot motion estimates through /odom. The LiDAR driver supplies /scan and the camera publishes /camera/image_raw; image transport makes a compressed stream available to the processing machine. This division keeps time-sensitive base-control and sensor interfaces close to the robot while allowing perception and mission logic to run on the Ubuntu machine.

2.2 Data and control flow

During map construction, SLAM Toolbox combines LiDAR observations with odometry and TF information to estimate the robot's trajectory and produce an occupancy-grid map on /map. The map is saved as an image and metadata pair, then reused in the autonomous stage. The navigation stack loads the saved map through the ROS 2 map server and uses Adaptive Monte Carlo Localization (AMCL) to estimate the robot pose.

After localization, the mission controller generates reachable observation locations from the saved occupancy-grid map and selects destinations according to their expected coverage of unexplored areas. Navigation goals are sent to the custom A* navigator through /astar/command. The navigator computes collision-aware paths, generates velocity commands through /cmd_vel and reports navigation progress through /astar/status.

In parallel, the camera and YOLO detector provide object detections. When the selected target is recognized, the mission controller temporarily suspends the coverage-based search and directs the robot to a second observation position. The target's bearings from different camera positions are combined with TF transformations to estimate its coordinates through monocular triangulation. Once a consistent target position is confirmed, the mission controller commands the A* navigator to return to the recorded starting position.

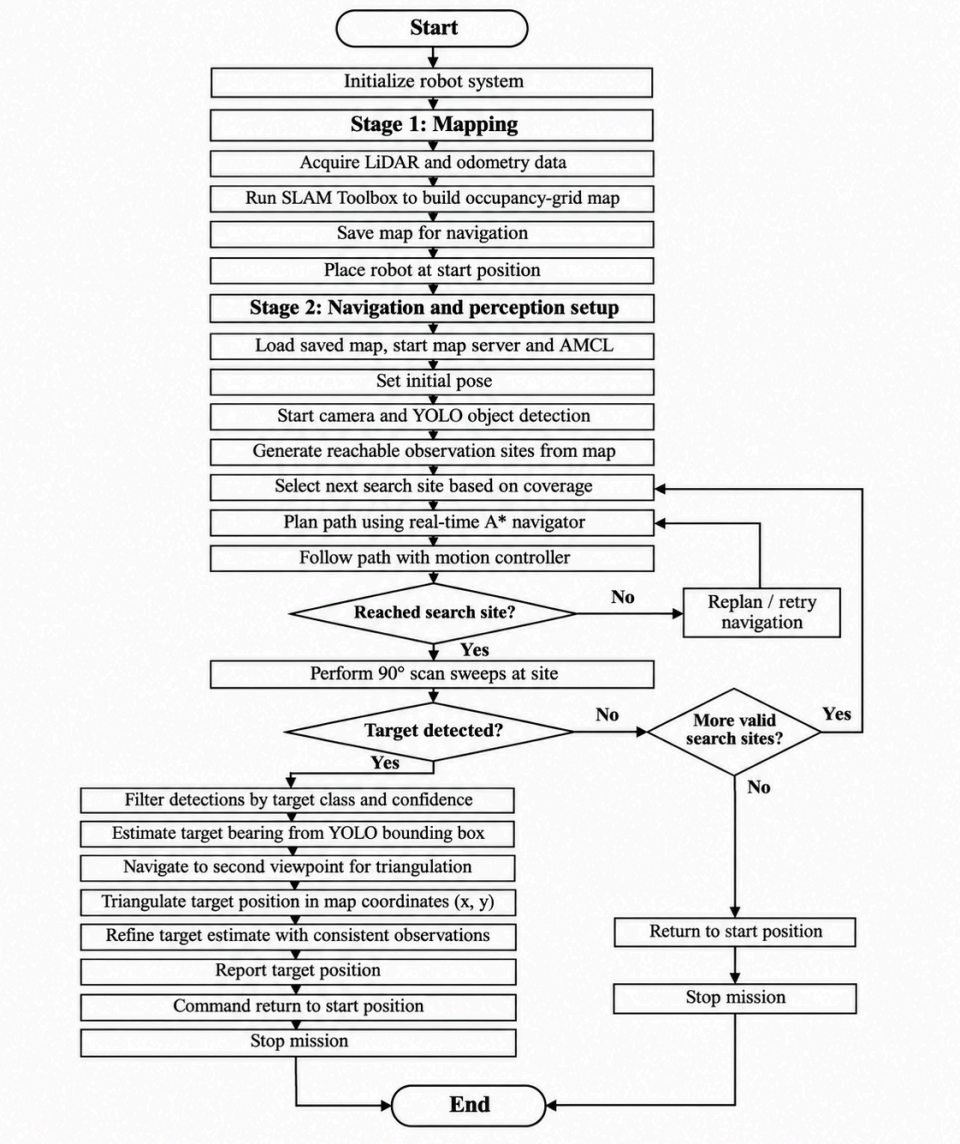


Figure 1. Workflow of the EE3033 autonomous mapping, navigation and target-localization system.

2.3 ROS 2 nodes, topics and transforms

The principal ROS 2 data interfaces are /scan and /scan_fixed for laser measurements, /odom for odometry, /map for the occupancy grid, /cmd_vel for motion control, /yolo/detections for object recognition, and /astar/command, /astar/status and /astar_path for navigation commands, feedback and route visualization. The mission controller also publishes candidate search locations through /ee3033/search_sites and estimated target coordinates through /ee3033/target_map, together with mission state and report topics.

The map frame provides the global coordinate reference for navigation and target localization. AMCL maintains the transformation from map to odom_combined, while robot odometry provides the transformation from odom_combined to base_footprint. Static transformations describe the positions and orientations of the camera, LiDAR and other sensors relative to the robot.

For each valid object detection, TF provides the camera's position and orientation in the map frame at the corresponding observation timestamp. This allows target bearings obtained from different camera positions to be expressed in a common coordinate system and combined through triangulation. The same coordinate reference is used for navigation goals and the return to the recorded starting position.

3. Mapping and localization

3.1 LiDAR processing

The lslidar_driver_n10p package acquires the two-dimensional laser scans. A scan-resampling node supplies processed LaserScan messages for SLAM Toolbox. As the balancing car moves around the sandbox during the permitted mapping stage, SLAM Toolbox combines these measurements with the base odometry to form a two-dimensional occupancy grid. RViz2 is used to view walls, free space and the robot pose so that mapping coverage can be assessed before the map is saved.

The mapping launcher, mapping.py, starts the required ROS 2 processes, RViz2 and keyboard control. When map construction is complete, the ROS 2 map-saving tool writes the occupancy-grid image and corresponding YAML metadata. This creates the fixed environment representation used in autonomous navigation.

3.2 Map representation and threshold handling

The saved occupancy grid must distinguish observed free space from unknown regions outside the mapped area. This distinction affects both autonomous goal selection and obstacle avoidance. In the recorded mapping setup, the map-saving utility wrote free_thresh: 0.25 into the YAML metadata; when the map was reloaded, some grey pixels representing unknown space could be classified as free. The map-saving script was modified to set free_thresh: 0.19 in the saved YAML file, so those unknown pixels remain unobserved when the navigation map is loaded.

The search-site generator excludes unknown cells and cells close to mapped obstacles. Keeping unknown space out of the traversable map prevents it from selecting destinations beyond the mapped area and gives the route planner an obstacle representation consistent with what the LiDAR has actually observed.

3.3 Localization on the saved map

After mapping, the robot is placed at the designated start location. The known-map bring-up configuration loads the saved map and starts AMCL, which uses LiDAR readings and odometry to estimate pose on the map. An initial pose is provided through the set-pose command. The mission and A* nodes then use the map-to-base_footprint TF relationship to obtain the robot's estimated position during autonomous movement. Using localization against the saved occupancy grid keeps search destinations and target coordinates in the same coordinate system. Accurate TF and pose estimates remain necessary because drift would affect route following and the final target-position report.

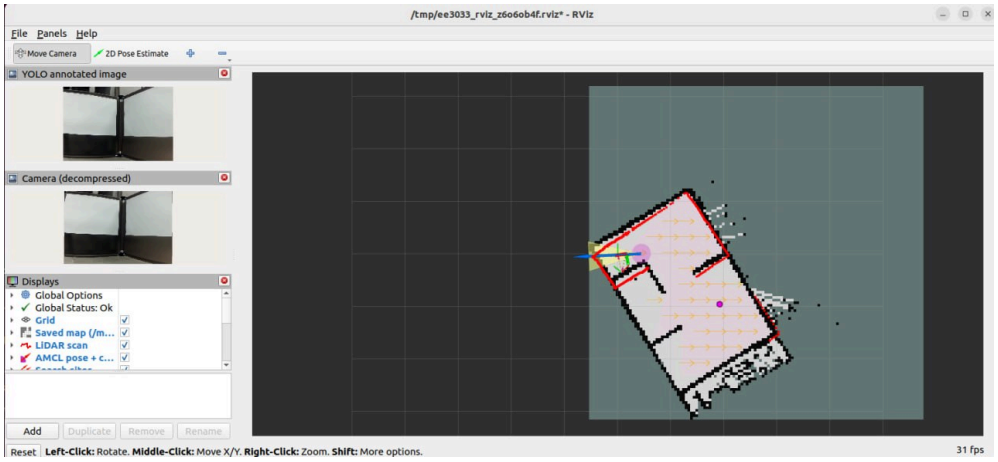


Figure 2. RViz visualization of the saved occupancy-grid map, robot localization and camera image streams

4. Autonomous navigation and search

4.1 A* route planning and clearance

The custom A* navigator uses an occupancy-grid map to generate collision-aware paths. To support navigation through narrow passages, the planner applies a hard obstacle-inflation distance of 0.075 m while introducing a soft traversal cost within 0.20 m of obstacles. This allows narrow free-space routes to remain available while encouraging the planner to select paths closer to the centre of wider passages. The A* algorithm considers eight-connected grid movements, accounts for diagonal travel distances, and prevents diagonal corner-cutting through occupied cells. The resulting route is published through /astar_path and updated using new LiDAR observations to account for obstacles not represented in the saved map.

4.2 Movement control and safeguards

The path follower converts the next part of the planned route into forward and angular velocity commands for the balancing-car base. The configured maximum linear and angular speeds are 0.10 m/s and 0.35 rad/s respectively. The software also applies a forward-obstacle guard, checks the availability of recent LiDAR data and a valid pose, and limits navigation using progress and goal timeouts. When required control information is unavailable, the controller sends a zero-velocity command.

The movement controller incorporates front and rear obstacle monitoring to support safe navigation in confined spaces. A 0.20 m wide monitoring corridor is used to detect obstacles directly along the robot's direction of travel, reducing unnecessary stops caused by nearby side walls. Forward obstacles within 0.45 m trigger movement adjustments and speed reduction, while obstacles within 0.20 m inhibit forward movement. When forward movement is obstructed, the controller can reverse if sufficient clearance is available behind the robot. Live obstacles remain included in collision checking near navigation goals. The controller also monitors LiDAR freshness, localization availability and navigation progress, issuing zero-velocity commands when the required movement conditions are not satisfied.

4.3 Coverage-based search strategy

The mission controller autonomously selects observation sites from the saved occupancy-grid map instead of following a predetermined route. It identifies free cells with sufficient obstacle clearance and restricts candidate locations to the region connected to the robot's starting position. The grid_planner component distributes observation sites across reachable free space using configurable spacing and a limit on the total number of candidates. Observation locations are selected with additional obstacle clearance to support stationary camera scanning. Narrow passages may remain available for navigation, while observation goals are preferentially placed in more open areas.

The search algorithm estimates line-of-sight coverage from each candidate location by ray casting through free cells. It records which cells have already been observed within the camera field of view. The next observation site is chosen according to its expected unseen-area coverage relative to the travel distance, while locations with insufficient new coverage are skipped. At a selected site, the controller issues 90-degree rotation commands to inspect other directions. This reduces repeated observations and concentrates movement on parts of the map that have not yet been viewed. When the target is initially detected, the mission controller temporarily suspends the coverage-based search and initiates a target-localization procedure. It selects an accessible second viewpoint with sufficient separation from the initial observation position. The robot navigates to this location to obtain a different viewing angle for triangulation. If the target cannot be reliably localized, the controller may attempt another viewpoint before resuming the search.

5. Camera perception and target localization

5.1 Camera processing and object detection

The v4l2_camera node acquires frames from the USB camera and image transport makes compressed images available to the Ubuntu virtual machine. The camera is configured at 640 × 480 pixels. The capture rate was capped at 10 frames per second to reduce data and processing load relative to the camera's original 30-frame-per-second stream; the recorded YOLO inference rate was substantially lower than the original camera rate.

The active detector uses a YOLO26s ONNX model with CPU inference. The model loaded 80 COCO class names and produced detections for objects including bottles, people and chairs in the recorded perception runs. It operates separately from camera capture and the navigation controller so that the software can continue managing the robot pose and movement without depending on the detector for every control cycle.

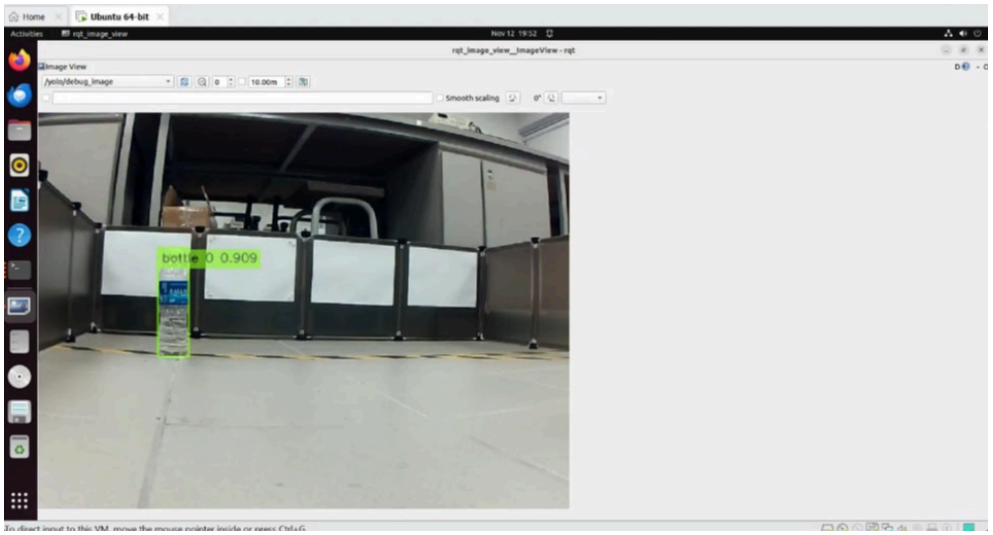


Figure 3. YOLO object detection showing a bottle identified.

5.2 Target selection and detection confirmation

The target object is not known when mapping begins. The desired class is therefore held in a configurable target_class setting that can be changed after the object is announced. This avoids hard-coding a single object category into the perception pipeline. The detector must still have a trained class corresponding to the selected object.

The detection pipeline applies a confidence threshold to identify potential target observations, while the mission controller performs additional filtering before accepting a detection for localization. Initial target recognition triggers the acquisition of observations from another camera position rather than an immediate coordinate report. The collected observations are evaluated using their camera poses and viewing directions. Triangulated estimates must satisfy geometric consistency requirements and are checked against the occupancy-grid map to reject invalid positions. Short interruptions in target detection are tolerated, and additional observations are used to refine the estimated coordinates before final confirmation.

5.3 Estimating a target position in map coordinates

The target-localization system uses monocular vision triangulation to estimate the target's position without requiring LiDAR distance measurements. YOLO detects the selected object and provides the horizontal centre of its bounding box. This pixel coordinate is converted into a bearing relative to the camera using the calibrated horizontal field of view.

The robot's camera position and orientation are obtained through ROS 2 TF transformations. Each detection therefore defines a viewing ray originating from the camera's position in the map coordinate frame. However, a single observation provides only the direction of the target and cannot determine its distance. To estimate the target position, the robot autonomously moves to a second observation position. This movement creates a baseline between the camera viewpoints, allowing the target to be observed from a different angle. The intersection of the viewing rays provides an estimate of the target's location. Multiple observations are subsequently combined using a least-squares approach to reduce measurement errors.

Bearing from image coordinate	Viewing ray	Target estimate
$\theta_i = \psi_i + \left(\frac{u_i - W/2}{W}\right) F$	$P_i(t) = C_i + t \begin{bmatrix} \cos\theta_i \\ \sin\theta_i \end{bmatrix}$	Two observations from different camera positions estimate the target from the intersection of their rays. Multiple observations can be combined using least-squares estimation.
C_i = camera position, ψ_i = camera heading, u_i = bounding-box centre, W = image width, F = calibrated horizontal field of view, t = distance parameter.		

Figure 4. Mathematical formulation of camera-based bearing estimation and target-position triangulation.

The estimated coordinates are validated against the occupancy-grid map to reject physically inconsistent positions, including estimates on mapped obstacles or in unknown regions. The system also checks the consistency of repeated observations before confirming the target position. Once confirmed, the estimated coordinates are published in the map frame and included in the mission report. The accuracy of this method depends on camera calibration, the separation between observation positions, the angular difference between viewing rays and the accuracy of the robot's localization.

6. Mission coordination and deployment

6.1 Stage-specific launch structure

The software is organized around two main Python entry points. The mapping.py launcher starts the SLAM and visualization stage and provides map-saving controls. The ee3033.py launcher coordinates perception, known-map navigation, initial-pose publication, status checking, mission start and shutdown. The navigation process starts the robot-side A* controller and map-localization services, while the mission controller runs on Ubuntu and communicates through ROS 2 topics. The final navigation implementation is maintained in EE3033_Nav_v10, while the mapping system is maintained in EE3033_Mapping_v7.4. Both implementations are available in the GitHub repository provided in Appendix A.

The perception processes are managed separately from navigation, allowing the camera and detector to be started before autonomous movement. The launcher uses SSH to control robot-side processes inside Docker, with connection and shutdown handling configured to avoid blocked interactive password prompts. The mission remains disarmed until the start command is issued.

6.2 Mission execution and return control

At mission start, the controller records the robot's initial pose and generates reachable observation locations from the occupancy-grid map. The custom A* navigator is commanded to visit selected locations while the mission controller updates the estimated visual coverage of the environment. At each observation site, the robot performs rotational scanning to search for the selected target class.

When a potential target is recognized, the controller suspends the normal search procedure and selects another reachable viewpoint for triangulation. The robot navigates to this position and acquires further camera observations. The resulting target estimates are evaluated for geometric consistency and agreement with the mapped environment. Once a valid position is confirmed, the controller reports the target coordinates and commands the robot to return to its recorded starting position.

If the target cannot be detected or reliably localized, the controller can resume exploration or attempt another observation position. When the search is exhausted or further navigation is unsuccessful, the system attempts to return to its starting position without reporting an unconfirmed target location. Navigation retries, mission-state transitions and shutdown conditions are handled by the mission controller.

7. Experimental results and performance observations

7.1 Mapping and autonomous navigation tests

Initial testing of the map-based search-site generator produced only one valid candidate location. Subsequent adjustments to map threshold handling and planning clearance were followed by a new mapping session in E4A Room 3. The resulting map generated 39 valid observation candidates. In a test without object detection, the robot navigated to valid candidate locations and returned to its starting position. Some movement drift was observed during the navigation test.

The increase in valid observation locations following the map adjustments indicates that occupancy-grid interpretation directly affected autonomous search-site generation. The navigation tests also demonstrated that the system could execute paths to selected observation locations and return to its starting position. However, movement drift remained observable during testing, and quantitative repeat-trial success rates were not established.

7.2 System integration and perception observations

During integration, SSH password prompts initially caused perception shutdown timeouts. The launcher was subsequently modified to prevent blocked interactive prompts, resolving the shutdown issue. A leftover keyboard-control node from mapping also prevented navigation startup because it could send velocity commands at the same time as the A* controller. Removing the stale process allowed the navigation stack to restart.

The camera initially produced frames at a higher rate than the CPU-based YOLO detector could process. The capture rate was reduced to lower image-stream load, while perception remained separate from the robot-side movement controller. These adjustments addressed communication and processing bottlenecks but do not by themselves establish the final camera-detection throughput under full mission load.

An additional integration issue affected the YOLO debug image displayed in RViz. Although object detections were being published successfully, the debug visualization subscribed to the tracking topic while object tracking was disabled. The visualization configuration was modified to subscribe directly to the raw detection topic, restoring the annotated image output. Subsequent testing confirmed that the debug visualization could display a detected bottle with its bounding box and confidence score.

7.3 Target localization and end-to-end mission evaluation

A separate integrated autonomous mission was conducted to evaluate object detection, camera-based triangulation, navigation and return-to-start functionality. The system generated 37 candidate observation sites and began searching for the selected target class, bottle. Upon detecting the target, the mission controller suspended the coverage-based search and commanded the robot to obtain observations from a second camera viewpoint. During the initial localization attempt, the system generated several triangulated position estimates but could not obtain sufficient confirmation. The mission controller resumed the coverage-based search and subsequently detected the target again. Additional observations produced a consistent position estimate, which was confirmed and refined before the final report.

The final reported target position was (1.919, −0.954) m in the map coordinate frame, based on six accepted samples. The robot then autonomously navigated back to its recorded starting position. The A* controller reported successful arrival within 0.14 m of the return goal. The autonomous search, target-localization and return mission took 87.1 seconds, excluding the separate mapping stage. The reported return distance was based on the robot's localization estimate rather than an independent physical measurement. This experiment demonstrated the integrated operation of target detection, viewpoint repositioning, triangulation, failure recovery and autonomous return. However, the actual target coordinates were not recorded in the available results, so the target-localization error could not be determined. The reported return distance was based on the navigation controller's localization estimate rather than an independent physical measurement.

8. Limitations, failure cases and possible improvements

8.1 Motion drift and localization reliability

The balancing platform exhibited movement drift during navigation tests. Separate motion observations also indicated unwanted clockwise rotation, which can affect heading control and the distance actually travelled between camera viewpoints. These effects can reduce path-following accuracy and introduce error into triangulation if the estimated camera poses do not match the physical motion. Separately evaluating the base yaw-feedback controller and the A* path follower would help distinguish low-level movement instability from navigation-planning errors. Comparing commanded motion against measured physical displacement would also provide a basis for improving movement accuracy.

Target triangulation and navigation both depend on a consistent map-to-robot transform. A missing camera transform, incorrect odometry-frame selection or stale localization estimate can prevent observations from being placed correctly in map coordinates. A future implementation could monitor localization update age and confidence, rather than checking only that a transform exists.

8.2 Camera-only target-location limitations

The camera-only method cannot determine range from a single stationary viewpoint. It requires enough translation to create parallax, and errors grow when two observation rays are nearly parallel. Motion restrictions, repeated missed detections and incomplete target visibility can therefore prevent successful localization even when the object class has been recognized. An improved viewpoint selector could use predicted triangulation geometry and observed pose uncertainty to choose the next reachable location.

The method also depends on camera field-of-view and yaw calibration, correct timestamps and stable detection boxes. Image latency during robot rotation can bias bearings, while targets near mapped walls may be rejected by the wall-consistency test. Improved synchronization, stable observation pauses and calibrated wall-rejection criteria could reduce these errors. The current approach assumes a stationary target and may not distinguish reliably between multiple objects of the same class.

The perception pipeline also exhibited variable detection throughput, with occasional intervals between detections exceeding the configured target-loss timeout. Such interruptions can interfere with the accumulation of consistent observations during triangulation. During integrated testing, the initial viewpoint-acquisition attempt did not produce a confirmed target position, requiring the controller to resume exploration before a subsequent successful localization attempt. Increasing tolerance to temporary detection losses and improving inference consistency could reduce unnecessary viewpoint retries.

8.3 Obstacle sensing and system integration

The two-dimensional LiDAR observes obstacles mainly at its scan height. Small objects below that plane may therefore be visible to the camera but absent from collision checks. Consequently, detecting a target through the camera does not guarantee that the corresponding travel corridor is free of obstacles. Additional low-level obstacle sensing or a more conservative target keep-out region could reduce the risk of collision with objects below the LiDAR scanning plane.

A* navigation is designed to be the active movement controller, while unused Nav2 components can remain present in the graph. The deployment process checks for conflicting teleoperation nodes to avoid simultaneous commands to /cmd_vel. Future work could add stronger command-source arbitration, explicit monitoring of stale processes and a global mission timeout to limit repeated acquisition attempts.

System reliability was also affected by insufficient storage on the Raspberry Pi. Accumulated crash dump files filled the SD card, preventing Docker from starting normally and interrupting camera operation. Removing the accumulated files and configuring the system to prevent further crash dump accumulation restored available storage. However, the underlying cause of the earlier process crashes had not been established, leaving a potential reliability limitation.

Appendix A. GitHub Source Code Repository

The complete source code for the EE3033 balancing-car project is available in the following GitHub repository: https://github.com/twoipe/ee3033-balancing-car

Appendix B. AI Declaration

Our group used AI tools, including ChatGPT, OpenAI Codex and Claude, to assist with code development, debugging, troubleshooting, parameter calibration, idea generation, explaining technical concepts, and refining the structure, language and formatting of this report. We reviewed and adapted the AI-generated content and tested the implementation on the physical robot. Our group takes full responsibility for the content and quality of the submitted work.